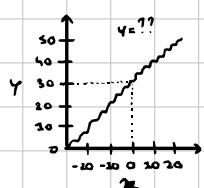
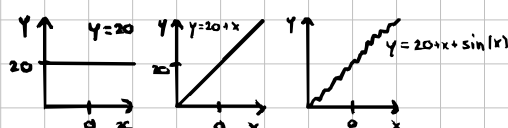


Gradient Boosting

Additive modeling



How can we decompose the function?
Suppose it's made of multiple simple terms.



So for target function $F(x)$ use sum of subfunctions.

$$F_M(x) = \sum_{m=1}^M \beta_m(x)$$

Gradient Boosting

$$\hat{y} = \beta_0(x) + \sum_{m=1}^M \Delta_m(x) = F_M(x)$$

Start with base model to give first prediction $\beta_0(x)$ given feature vector x .

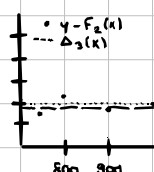
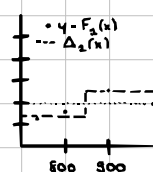
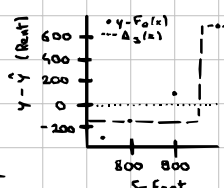
Gradually nudge $F_m(x)$ model by adding models $\Delta_m(x)$.

After first approximation we get difference $y - \beta_0(x)$ called **residual vector**.

↳ residual vector as direction + magnitude to correct prediction

↳ train $\Delta_m(x)$ on $y - F_{m-1}(x)$

Optimising L_2 loss



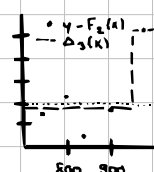
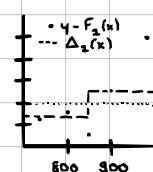
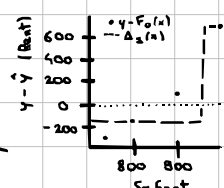
Training on $y - F_{m-1}(x)$ can make $\Delta_m(x)$ chase outliers

Optimising L_1 loss

Other approach is to train $\Delta_m(x)$ using $\text{sign}(y - F_{m-1}(x))$.

↳ $\Delta_m(x)$ will try to predict the median residual

↳ Makes the model optimise for MAE / L_1 loss function



Training on median will avoid chasing outliers

Measuring Model Performance

Using loss or cost function, $L(y, \hat{y})$ or $L(y_i, \hat{y}_i)$.

Loss across N observations:

$$L(y, F_M(x)) = \frac{1}{N} \sum_{i=1}^N L(y_i, F_M(x_i))$$

The Mean Squared Error (MSE) or L_2 loss function:

$$L(y, F_M(x)) = \frac{1}{N} \sum_{i=1}^N (y_i - F_M(x_i))^2$$

The Mean Absolute Error (MAE) or L_1 loss function:

$$L(y, F_M(x)) = \frac{1}{N} \sum_{i=1}^N |y_i - F_M(x_i)|$$

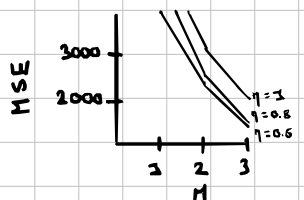
Mean is exact min. of L_2 :

$$\begin{aligned} \min_y \sum_{k=1}^n (r_k - y)^2 \\ \Leftrightarrow \frac{\partial}{\partial y} \sum_{k=1}^n (r_k - y)^2 &= 0 \\ \Leftrightarrow \sum_{k=1}^n 2(r_k - y)(-1) &= 0 \\ \Leftrightarrow \sum_{k=1}^n (r_k - y) &= 0 \\ \Leftrightarrow \sum_{k=1}^n r_k - ny &= 0 \\ \Leftrightarrow y &= \frac{1}{n} \sum_{k=1}^n r_k \end{aligned}$$

⚠ Same for median and L_1 loss

Learning Rate

$$F_{m+1}(x) = F_m(x) + \eta \Delta_{m+1}(x)$$



Learning rate ("shrinkage") helps reduce overfitting.

We can use grid search for parameter tuning.

Also here learning rate is constant but could be reduced across M for more precise calibration.

Gradient Descent

To find how to minimise loss $\beta(x, y)$, we compute the direction vector

from partial derivatives of $\beta(x, y)$: $\frac{\partial \beta(x, y)}{\partial x}$, $\frac{\partial \beta(x, y)}{\partial y}$.

Stacking derivatives in a vector gives us the gradient: $\nabla \beta(x, y) = \begin{bmatrix} \frac{\partial \beta(x, y)}{\partial x} \\ \frac{\partial \beta(x, y)}{\partial y} \end{bmatrix}$ (direction of steepest increase)

⚠ Chasing the direction vector in a GBM is chasing the (negative) gradient of a loss function via gradient descent.

MSE function gradient

$$L(y, F_M(x)) = \frac{1}{N} \sum_{i=1}^N (y_i - F_M(x_i))^2$$

$$L(y, \hat{y}) = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

$$L(y, \hat{y}) = \sum_{i=1}^N (y_i - \hat{y}_i)^2 \quad (\beta(x) \text{ and } c\beta(x) \text{ have same min.})$$

$$\frac{\partial}{\partial \hat{y}_i} L(y, \hat{y}) = \frac{\partial}{\partial \hat{y}_i} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

$$= \frac{\partial}{\partial \hat{y}_i} (y_i - \hat{y}_i)^2 \quad (\text{only prediction } i \text{ impacts observation } i)$$

$$\nabla_{\hat{y}_i} L(y, \hat{y}) = -2 \underbrace{(y_i - \hat{y}_i)}_{\text{residual vector}}$$

MAE function gradient

$$L(y, \hat{y}) = \sum_{i=1}^N |y_i - \hat{y}_i|$$

$$\frac{\partial}{\partial \hat{y}_i} L(y, \hat{y}) = \frac{\partial}{\partial \hat{y}_i} \sum_{i=1}^N |y_i - \hat{y}_i|$$

$$= \frac{\partial}{\partial \hat{y}_i} |y_i - \hat{y}_i|$$

$$\nabla_{\hat{y}_i} L(y, \hat{y}) = \underbrace{\text{sign}(y_i - \hat{y}_i)}_{\text{sign vector}} (-1)$$